KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
https://jti.polinema.ac.id

Mata Kuliah	:	PEMOB
Program Studi	:	D4 – Teknik Informatika
Semester	:	5
Kelas	:	TI - 3E
NIM	:	244107020207
Nama	:	Adam Bahy Maulana
Jobsheet Ke-	:	7

LAPORAN JOBSHEET

Praktikum Ke-1 : Audit Layer Project Lama

<table>
<tr><th>Langkah</th><th>Jawaban/Deskripsi</th></tr>
<tr><td>1 : Petakan berkas ke layer</td><td>Project yang diaudit adalah campus_notify milik Minggu 6 (autentikasi, penyimpanan token, dan FCM). Susunan awalnya memakai empat folder mendatar, yaitu data, providers, pages, dan messaging, ditambah router serta routes di akar lib.

Tabel 1. Pemetaan berkas lama ke layer
<table><tr><th>Berkas</th><th>Layer saat ini</th><th>Masalah</th></tr><tr><td>pages/home_page.dart</td><td>presentation</td><td>mengimpor data/token_store.dart dan memakai TokenStore.mask langsung</td></tr><tr><td>pages/login_page.dart</td><td>presentation</td><td>mengimpor data/api_errors.dart untuk menerjemahkan galat</td></tr><tr><td>providers/auth_provider.dart</td><td>presentation (state)</td><td>membuat AuthRepository() di dalam provider; kontrak dan implementasi belum dipisah</td></tr><tr><td>data/auth_repository.dart</td><td>data</td><td>entity Session, kontrak, dan implementasi menumpuk dalam satu berkas</td></tr><tr><td>data/token_store.dart</td><td>data</td><td>aman, tetapi dibaca langsung oleh halaman</td></tr><tr><td>messaging/deeplink.dart</td><td>infrastruktur</td><td>fungsi murni tersimpan di folder messaging sehingga helper murni tersebar</td></tr></table></td></tr>
<tr><td>2 : Tandai tiga pelanggaran klasik</td><td>Tiga pencarian dijalankan pada project Minggu 6. Dua pencarian pertama tidak menghasilkan apa pun karena Minggu 6 sudah menyembunyikan Dio di balik provider, sehingga widget tidak memanggil jaringan secara literal.

<pre>$ rg -n "Dio\(|http\.|openDatabase|SharedPreferences\.getInstance|
FlutterSecureStorage" lib/pages lib/providers

(nol hasil)

$ rg -n "DateFormat|jsonDecode|\.toIso8601String" lib/pages lib/providers

(nol hasil)

$ rg -n "Repository\(|Dio\(BaseOptions" lib/pages lib/providers

lib/providers\auth_provider.dart:10: (ref) => AuthRepository(),</pre></td></tr>
</table>

Langkah	Jawaban/Deskripsi

	`$ rg -n "import '\.\./data" lib/pages`
	`lib/pages\login_page.dart:4:import '../data/api_errors.dart';`
	`lib/pages\home_page.dart:5:import '../data/token_store.dart';`
	Item refactor yang dihasilkan berjumlah tiga: instansiasi repository di luar batas layer, dua impor modul data oleh halaman, dan tidak adanya kontrak domain untuk sesi maupun pengumuman. Kondisi ini menyimpang dari dugaan modul yang menyebut widget memanggil Dio langsung; kebocoran yang sebenarnya terjadi adalah impor modul data oleh halaman, bukan pemanggilan objek Dio.

3 : Gambar struktur target	`lib/`
	`+-- core/failures.dart  Failure murni Dart`
	`+-- core/format.dart  tanggal, token, rute, pesan`
	`+-- features/auth/`
	`|  +-- domain/  entities/session.dart,`
	`|  |  repositories/auth_repository.dart`
	`|  +-- data/  datasources/{token_store,api_client,api_errors}.dart,`
	`|  |  repositories/auth_repository_impl.dart`
	`|  +-- presentation/  providers/auth_providers.dart,`
	`|  pages/login_page.dart`
	`+-- features/announcements/`
	`|  +-- domain/  entities/announcement.dart,`
	`|  |  repositories/announcement_repository.dart,`
	`|  |  usecases/get_announcements.dart`
	`|  +-- data/  models/announcement_model.dart,`
	`|  |  datasources/announcement_local_source.dart,`
	`|  |  repositories/announcement_repository_impl.dart`
	`|  +-- presentation/  providers/announcement_providers.dart,`
	`|  pages/{home_page,announcement_page}.dart`
	`+-- messaging/push_service.dart  infrastruktur FCM`
	`+-- router.dart, routes.dart`
	Snapshot sebelum refactor disimpan sebagai commit 492c6d1 dengan pesan chore: snapshot sebelum refactor Clean Architecture (week 7), sehingga struktur lama tetap dapat dibandingkan dan di-rollback.

Praktikum Ke-2 : Domain dan Data per Fitur

Langkah	Jawaban/Deskripsi
1 : Entity murni di domain	Dua entity dibuat tanpa impor Flutter dan tanpa fungsi pemetaan: Session untuk kredensial sesi dan Announcement untuk data pengumuman.

Langkah	Jawaban/Deskripsi

	`// lib/features/announcements/domain/entities/announcement.dart`
	`class Announcement {`
	`  const Announcement({`
	`  required this.id,`
	`  required this.title,`
	`  required this.summary,`
	`  required this.body,`
	`  required this.updatedAt,`
	`  });`
	
	`  final String id;`
	`  final String title;`
	`  final String summary;`
	`  final String body;`
	`  final DateTime updatedAt;`
	`}`

2 : Failure dan kontrak repository	Failure ditempatkan di lib/core karena dipakai semua fitur, sedangkan kontrak repository ditempatkan di domain masing-masing fitur. Status sukses dan gagal dikembalikan sebagai record Dart sehingga tidak ada exception yang bocor ke presentation.
	`// lib/core/failures.dart`
	`sealed class Failure {`
	`  const Failure(this.message);`
	`  final String message;`
	`}`
	`class LocalFailure extends Failure { const LocalFailure(super.message); }`
	`class NetworkFailure extends Failure {`
	`  const NetworkFailure(super.message);`
	`}`
	`class AuthFailure extends Failure { const AuthFailure(super.message); }`
	
	`// lib/features/announcements/domain/repositories/announcement_repository.dart`
	`abstract class AnnouncementRepository {`
	`  Future<({List<Announcement> items, Failure? failure})>`
	`  fetchAnnouncements();`
	`}`
	
	`// lib/features/auth/domain/repositories/auth_repository.dart`

Langkah	Jawaban/Deskripsi

	abstract class AuthRepository {
	Future<bool> hasSession();
	Future<String?> accessToken();
	Future<({Session? session, Failure? failure})> login({
	required String email, required String password});
	Future<({String? access, Failure? failure})> refresh(
	String refreshToken);
	Future<void> logout();
	}

3 : Model dan implementasi di data	Pemetaan bentuk luar hanya ada di model. Implementasi repository membaca sumber data contoh berbentuk JSON; komponen itulah yang kelak ditukar dengan Dio atau SQLite tanpa menyentuh domain maupun presentation.
	// lib/features/announcements/data/models/announcement_model.dart
	factory AnnouncementModel.fromJson(Map<String, Object?> json) {
	return AnnouncementModel(
	id: json['id']! as String,
	updatedAt: DateTime.tryParse(json['updated_at'] as String? ?? '')
	?? DateTime.fromMillisecondsSinceEpoch(0), ...);
	}
	
	//
	lib/features/announcements/data/repositories/announcement_repository_impl.dart
	try {
	final rows = await source.readAll();
	final items = rows
	.map((row) => AnnouncementModel.fromJson(row).toEntity())
	.toList();
	return (items: items, failure: null);
	} on Object catch (e) {
	return (items: const <Announcement>[],
	failure: LocalFailure('Gagal memuat pengumuman: $e'));
	}
	Sesi autentikasi mengikuti pola yang sama. AuthRepositoryImpl menjadi satu-satunya pemegang TokenStore, dan halaman tidak lagi membaca token mentah karena kontrak domain menyediakan metode accessToken().

4 : Use case satu operasi bisnis	// lib/features/announcements/domain/usecases/get_announcements.dart class GetAnnouncements {

Langkah	Jawaban/Deskripsi

	`const GetAnnouncements(this._repository);`
	`final AnnouncementRepository _repository;`
	
	`Future<({List<Announcement> items, Failure? failure})> call() ⇒`
	`_repository.fetchAnnouncements();`
	`}`
	Satu use case dipakai untuk daftar pengumuman karena operasi inilah yang berpotensi bertambah aturan, misalnya urutan, penyaringan, dan cache. Login, logout, serta perpanjangan token tidak diberi use case karena hanya meneruskan panggilan; alasannya ditulis pada docs/ai-challenge.md.

Praktikum Ke-3 : Presentation, Dependency Injection, dan Verifikasi

Langkah	Jawaban/Deskripsi

1 : Dependency injection dengan Riverpod	Berkas provider menjadi satu-satunya tempat penyambungan lapisan. Widget hanya melihat provider dan tidak membuat repository sendiri.
	`// lib/features/auth/presentation/providers/auth_providers.dart`
	`final tokenStoreProvider = Provider<TokenStore>((ref) => TokenStore());`
	
	`final authRepositoryProvider = Provider<AuthRepository>(`
	`(ref) => AuthRepositoryImpl(ref.watch(tokenStoreProvider)),`
	`);`
	
	`final apiClientProvider = Provider((ref) => buildApiClient(`
	`ref.watch(tokenStoreProvider),`
	`ref.watch(authRepositoryProvider)));`
	
	`final authStateProvider = AsyncNotifierProvider<AuthNotifier, bool>(`
	`AuthNotifier.new);`
	
	`//`
	`lib/features/announcements/presentation/providers/announcement_providers.dart`
	`final announcementRepositoryProvider = Provider<AnnouncementRepository>(`
	`(ref) => const AnnouncementRepositoryImpl(`
	`source: AnnouncementLocalSource()));`
	
	`final getAnnouncementsProvider = Provider<GetAnnouncements>(`
	`(ref) => GetAnnouncements(ref.watch(announcementRepositoryProvider)));`

<table>
<tr><th>Langkah</th><th>Jawaban/Deskripsi</th></tr>
<tr><td></td><td>

```
final announcementsProvider = FutureProvider<List<Announcement>>(
  (ref) async { ... });
```

</td></tr>
<tr><td>2 : Halaman hanya membaca state</td><td>

```
// lib/features/announcements/presentation/pages/home_page.dart
final announcements = ref.watch(announcementsProvider);
...announcements.when(
  loading: () => const [Center(child: CircularProgressIndicator())],
  error: (e, _) => [Text(messageOf(e))],
  data: (items) => [for (final item in items) _announcementCard(item)],
),
```

Kartu pengumuman tidak lagi diketik di dalam widget. Daftar dibaca dari provider, lalu halaman detail mengambil satu entri berdasarkan id dari state yang sama. Seluruh teks antarmuka dipertahankan sehingga perbandingan sebelum dan sesudah hanya menyangkut isi state.
</td></tr>
<tr><td>3 : Verifikasi dependency rule</td><td>
Tiga pemeriksaan dijalankan pada project hasil refactor dan keluaran lengkapnya disimpan di docs/verifikasi-grep.txt.

Tabel 2. Hasil pemeriksaan sterilitas

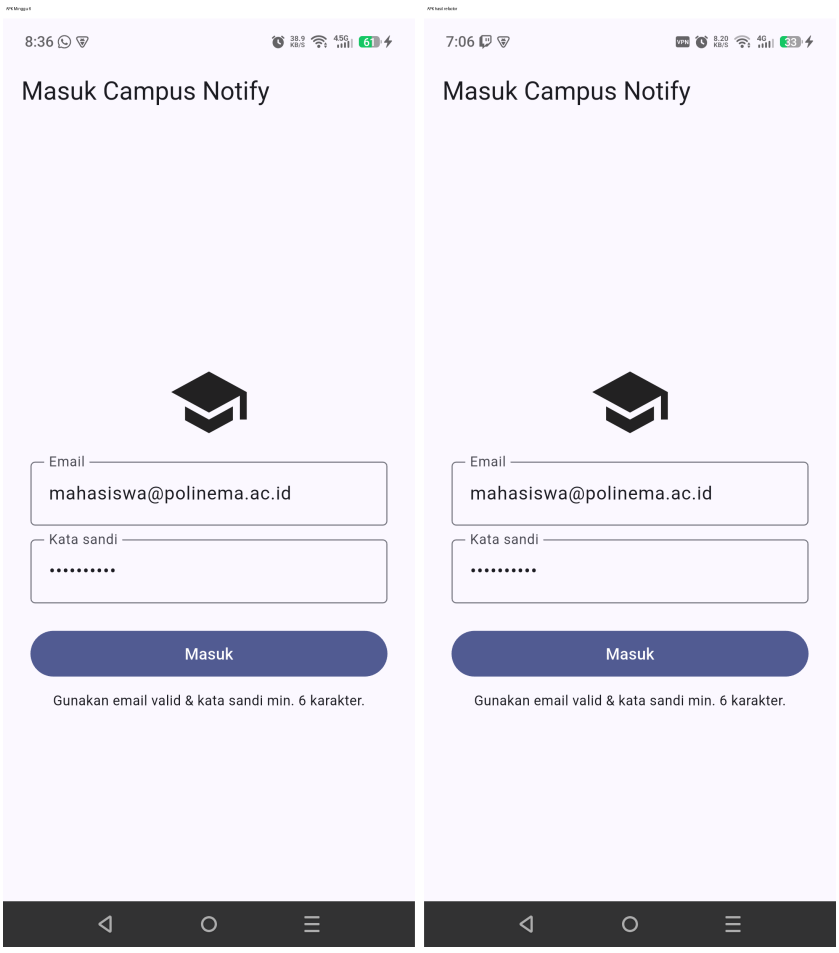
<table>
<tr><th>Pemeriksaan</th><th>Sasaran</th><th>Hasil</th></tr>
<tr><td>pola Dio, openDatabase, FlutterSecureStorage, SharedPreferences, jsonDecode</td><td>lib/features/*/presentation</td><td>nol hasil</td></tr>
<tr><td>impor package:flutter, dio, sqflite, firebase</td><td>lib/features/*/domain dan lib/core</td><td>nol hasil</td></tr>
<tr><td>pola RepositoryImpl(dan Dio(BaseOptions</td><td>lib/features/ */presentation/pages</td><td>nol hasil</td></tr>
<tr><td>pola yang sama di seluruh presentation</td><td>hanya berkas providers</td><td>dua hasil, keduanya titik wiring DI</td></tr>
</table>

```
$ flutter analyze
No issues found! (ran in 18.7s)


$ flutter test
00:00 +8: test/core_format_test.dart: messageOf memakai pesan Failure
00:00 +9: test/get_announcements_test.dart: GetAnnouncements meneruskan daftar
dari repository
00:00 +10: test/get_announcements_test.dart: GetAnnouncements meneruskan
failure tanpa melempar
00:00 +10: All tests passed!
```

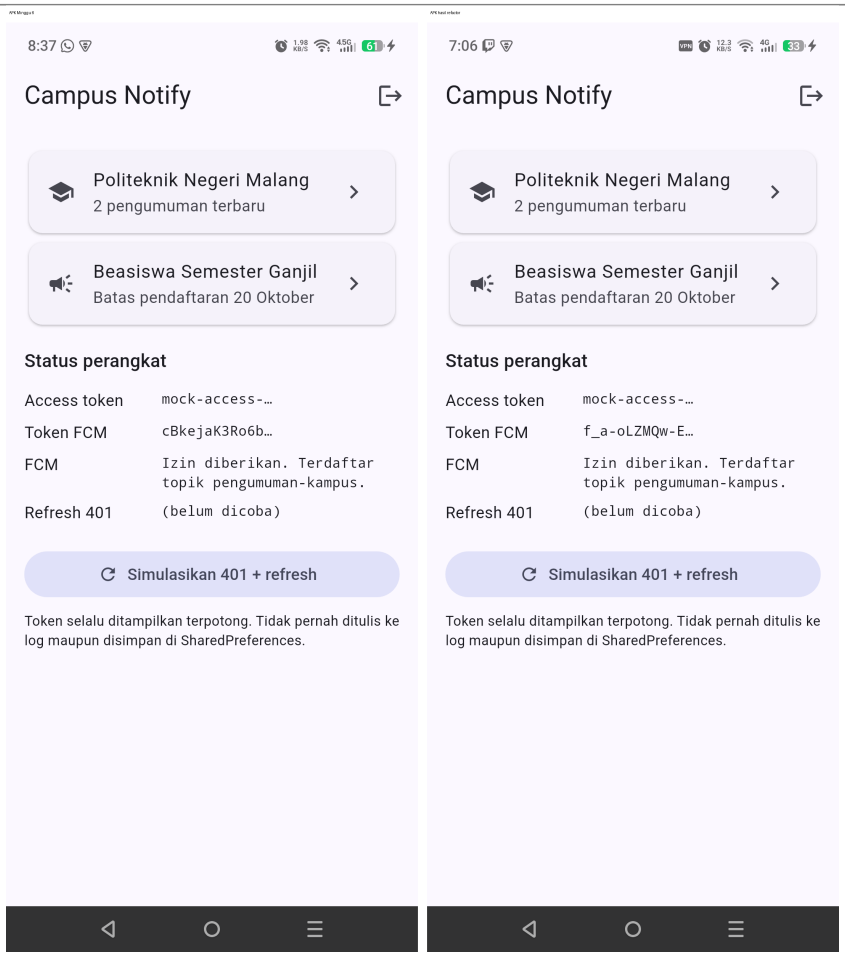
Sepuluh test lulus, termasuk dua test use case dengan repository palsu yang tidak menyentuh basis data maupun jaringan. Build rilis menghasilkan app-release.apk berukuran 53.087.187 byte dan dipasang pada perangkat Infinix resolusi 1080x2436.
</td></tr>
</table>

Langkah	Jawaban/Deskripsi
4 : Perbandingan tampilan	Uji manual dilakukan pada lima alur, yaitu login, beranda, detail pengumuman, simulasi 401, dan logout. Setiap alur direkam dua kali: APK Minggu 6 sebagai gambar kiri dan APK hasil refactor sebagai gambar kanan. Kedua tangkapan berukuran sama, 1080 x 2436 piksel, dan selisih piksel pada setiap alur di bawah satu persen: 0.21 % login, 0.34 % beranda, 0.92 % detail, 0.34 % simulasi 401, dan 0.22 % logout. Selisih itu berasal dari jam pada status bar dan cap waktu kartu, bukan dari susunan widget.



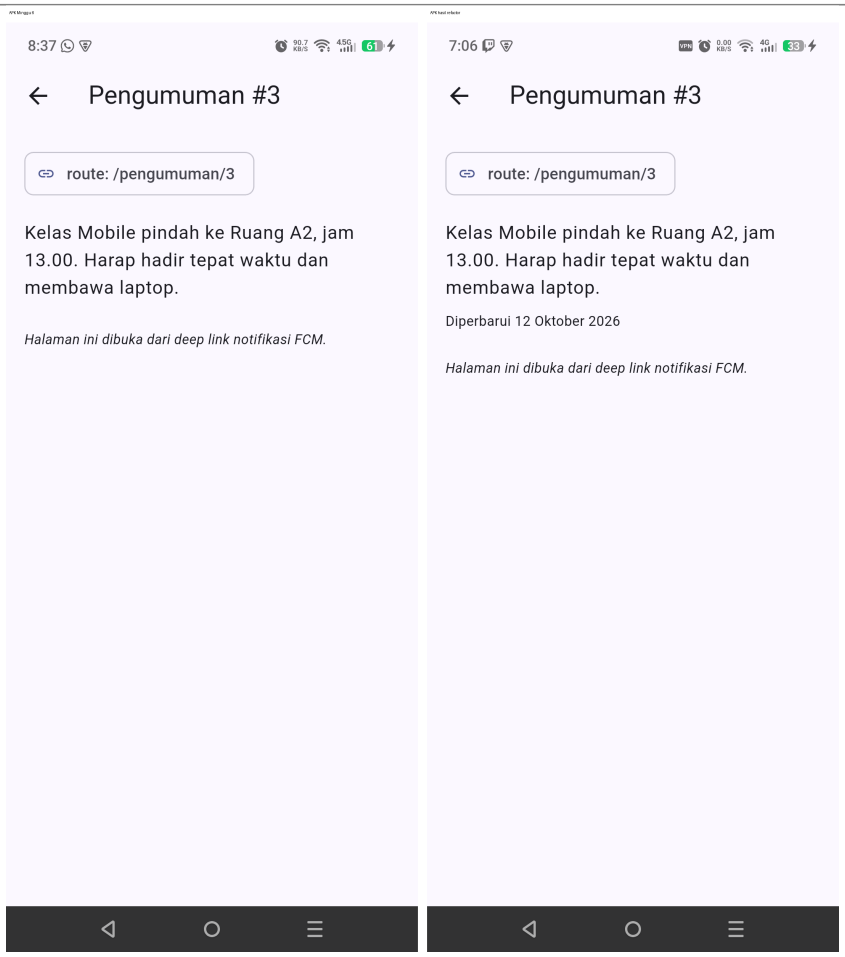
Gambar 1. Halaman login sebelum dan sesudah refactor; susunan widget, teks isian, dan tombol tidak berubah.

Langkah	Jawaban/Deskripsi



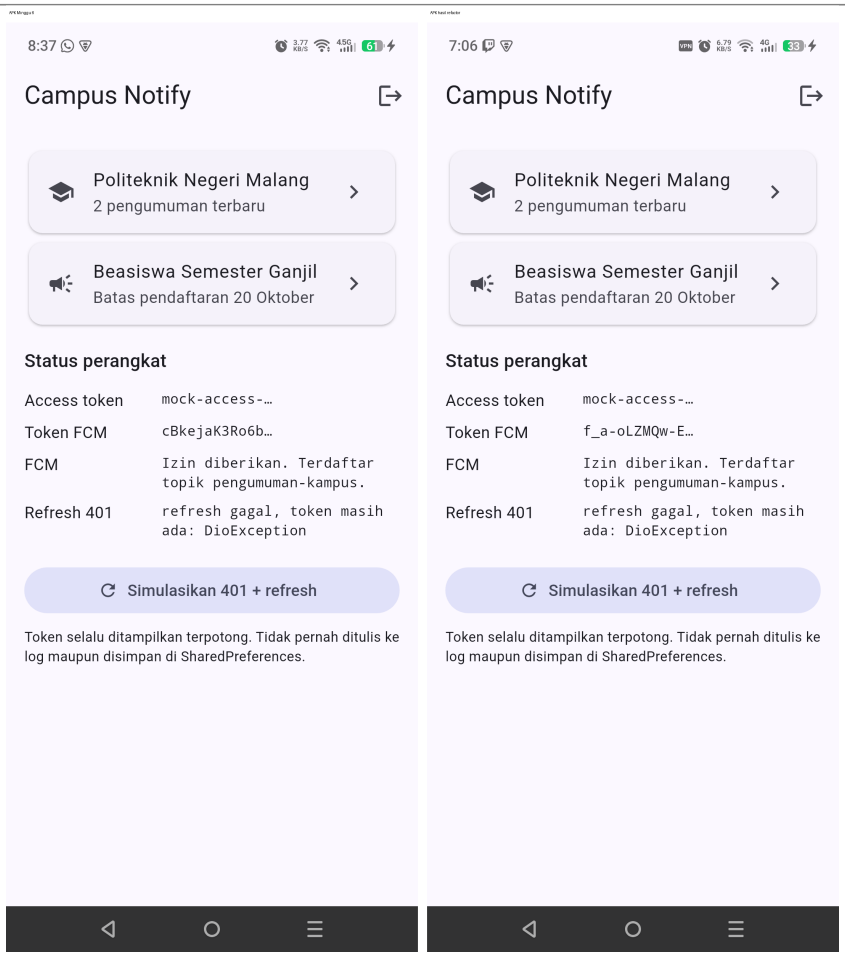
Gambar 2. Beranda dengan kartu pengumuman yang kini dibaca dari announcementsProvider; kolom status perangkat tetap menampilkan token terpotong.

Langkah	Jawaban/Deskripsi



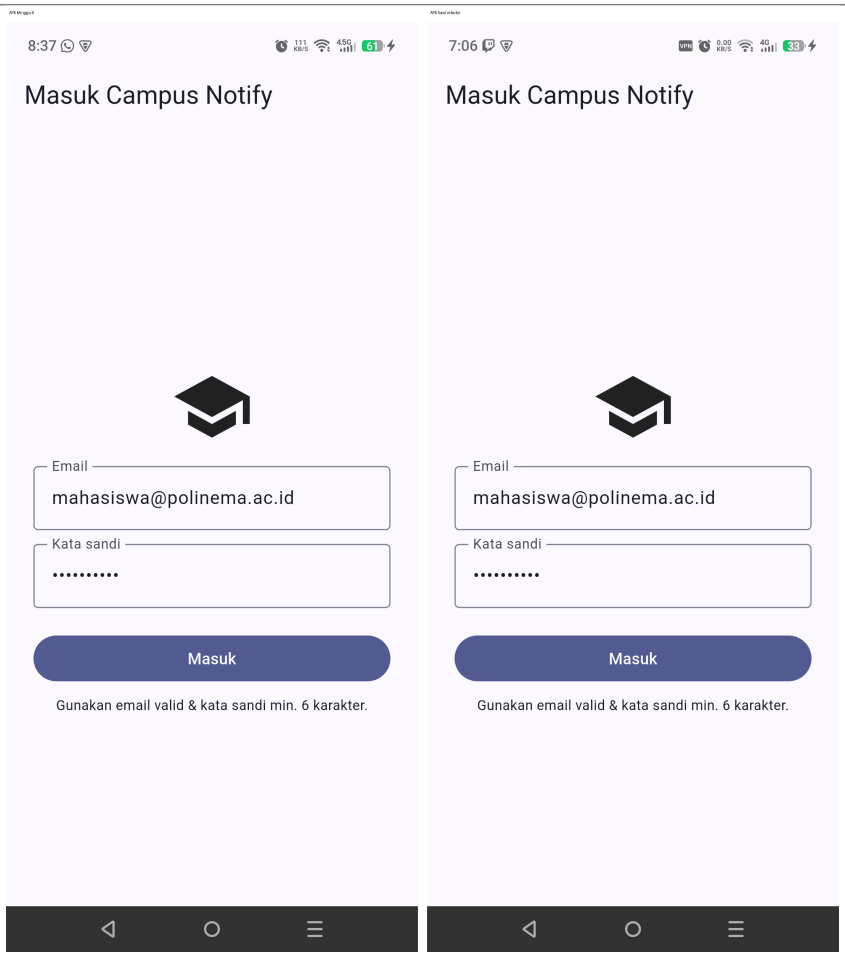
Gambar 3. Halaman detail pengumuman dengan baris tanggal hasil formatTanggal pada core/format.dart.

Langkah	Jawaban/Deskripsi



Gambar 4. Simulasi 401: interceptor menolak perulangan request dan membersihkan sesi ketika refresh ikut gagal.

Langkah	Jawaban/Deskripsi



Gambar 5. Logout mengembalikan aplikasi ke halaman login melalui redirect GoRouter.

Soal Praktikum : Refactor Menyeluruh, AI Challenge, dan Refleksi

Langkah	Jawaban/Deskripsi
1 : Refactor fitur kedua, auth	Fitur auth dipisah menjadi entity Session, kontrak AuthRepository, AuthRepositoryImpl, dan satu berkas provider. Notifikasi FCM dibiarkan menjadi infrastruktur pada lib/messaging karena isinya hanya pemanggilan SDK Firebase tanpa aturan bisnis.

Tabel 3. Tujuan baru tiap berkas lama

Berkas lama	Perlakuan	Berkas baru
data/token_store.dart	pindah	features/auth/data/datasources/token_store.dart
data/api_client.dart	pindah	features/auth/data/datasources/api_client.dart
data/api_errors.dart	pecah	api_errors.dart untuk peta DioException ke Failure, dan core/format.dart untuk messageOf

<table>
<tr><th>Langkah</th><th>Jawaban/Deskripsi</th></tr>
<tr><td></td><td>
<table>
<tr><td>data/auth_repository.dart</td><td>pecah</td><td>auth/domain/entities/session.dart,
auth/domain/repositories/auth_repository.dart,
auth/data/repositories/auth_repository_impl.dart</td></tr>
<tr><td>providers/auth_provider.dart</td><td>pindah</td><td>auth/presentation/providers/auth_providers.dart</td></tr>
<tr><td>pages/login_page.dart</td><td>pindah</td><td>auth/presentation/pages/login_page.dart</td></tr>
<tr><td>pages/home_page.dart dan pages/announcement_page.dart</td><td>pindah, isi diganti state</td><td>announcements/presentation/pages/</td></tr>
<tr><td>messaging/deeplink.dart</td><td>hapus dan lebur</td><td>core/format.dart pada fungsi routeFromMessage</td></tr>
</table>
</td></tr>
<tr><td>2 : Fungsi murni di core/format.dart</td><td>Empat fungsi murni dipusatkan di lib/core/format.dart agar dapat diuji tanpa widget.

```
String formatTanggal(DateTime waktu) ⇒
    '${waktu.day} ${_bulan[waktu.month - 1]} ${waktu.year}';
String maskToken(String? token) { /* 12 karakter pertama */ }
String routeFromMessage(Map<String, dynamic> data) { /* tambah '/' */ }
String messageOf(Object error) ⇒
    error is Failure ? error.message : /* potong prefix teknis */;
```

Fungsi tersebut diuji pada test/core_format_test.dart yang berisi delapan test, antara lain formatTanggal(DateTime(2026, 9, 27)) menghasilkan 27 September 2026 dan maskToken pada token 37 karakter menghasilkan mock-access- dengan elipsis.</td></tr>
<tr><td>3 : Failure sebelum mencapai UI</td><td>DioException diubah menjadi NetworkFailure atau AuthFailure pada features/auth/data/datasources/api_errors.dart, sedangkan kegagalan penyimpanan token diubah menjadi LocalFailure di dalam implementasi repository. Halaman hanya menerima teks melalui messageOf sehingga tipe Dio tidak pernah muncul di presentation.

```
// features/auth/data/datasources/api_errors.dart
case DioExceptionType.badResponse:
  final code = error.response?.statusCode;
  if (code == 401) return const AuthFailure(
      'Sesi berakhir. Silakan login ulang.');
  if (code != null && code >= 500) return const NetworkFailure(
      'Server sedang bermasalah.');
```
</td></tr>
<tr><td>4 : AI Challenge</td><td>Prompt, usulan awal, dan keputusan final AI Challenge dicatat pada docs/ai-challenge.md. Butir yang diubah atau ditolak dirangkum pada Tabel 4.

Tabel 4. Usulan AI dan keputusan final

<table>
<tr><th>Usulan AI</th><th>Keputusan</th><th>Alasan teknis</th></tr>
<tr><td></td><td></td><td></td></tr>
</table>
</td></tr>
</table>

<table>
<tr><th>Langkah</th><th>Jawaban/Deskripsi</th></tr>
<tr><td></td><td>
<table>
<tr><td>use case untuk login, logout, refresh, tambah, dan hapus</td><td>ditolak</td><td>operasi satu baris; tiga lapisan tambahan tidak menambah nilai uji</td></tr>
<tr><td>Either<Failure, T> dengan package fpdart</td><td>ditolak</td><td>record Dart sudah memberi status sukses dan gagal eksplisit tanpa dependensi baru</td></tr>
<tr><td>get_it dan injectable untuk DI</td><td>ditolak</td><td>Riverpod sudah dipakai state aplikasi; dua wadah DI membuat siklus hidup objek tidak jelas</td></tr>
<tr><td>satu AppRepository gabungan</td><td>ditolak</td><td>melanggar Interface Segregation karena pengumuman ikut bergantung pada token auth</td></tr>
<tr><td>push service dibungkus domain dan use case</td><td>ditolak</td><td>tidak memuat aturan bisnis; parsing rute murni dipindah ke core/format.dart agar teruji</td></tr>
<tr><td>interface di domain dan implementasi di data</td><td>diterima</td><td>domain tetap murni Dart sehingga use case diuji tanpa perangkat</td></tr>
</table>
</td></tr>
<tr><td>5 : Dependency rule terbukti</td><td>Jumlah berkas lapisan lib berubah dari 14 menjadi 22. Kenaikan delapan berkas berasal dari lapisan domain (lima berkas) dan core (dua berkas) yang sebelumnya tidak ada, ditambah satu pemisah sumber data. Tidak ada fitur baru yang dibuat selama refactor.

Arah dependensi terverifikasi: presentation mengimpor domain dan provider, data mengimpor domain, sedangkan domain hanya mengimpor domain dan core. Dua baris pertama Tabel 2 menjadi bukti tidak ada panah yang membalik, dan baris keempat memastikan pembuatan objek hanya terjadi di titik wiring.</td></tr>
<tr><td>6 : Refleksi</td><td>Mengapa interface repository harus tinggal di domain, bukan di data? Domain menetapkan kebutuhan dan data memenuhinya. Bila kontrak ditaruh di data, use case harus mengimpor modul yang membawa Dio atau secure storage, sehingga unit test domain ikut memerlukan perangkat dan jaringan.

Kapan use case benar-benar dibutuhkan? Ketika satu operasi menggabungkan lebih dari satu sumber data atau memuat aturan bisnis berupa validasi, urutan, dan sinkronisasi. Untuk CRUD satu baris, repository langsung ke notifier sudah cukup dan lebih murah.

Apa biaya over-engineering bagi tim kecil? Setiap lapisan tambahan menambah berkas yang harus dibuka saat membaca satu alur, memperbesar potensi konflik perubahan, dan memperlambat suntingan kecil. Biayanya sepadan ketika jumlah fitur bertambah hingga aturan bisnis tersebar, misalnya saat pengumuman memerlukan sinkronisasi luring.

Bagian mana dari usulan AI yang ditolak? Lima butir pada Tabel 4, yaitu use case untuk setiap operasi tunggal, fpdart, get_it, AppRepository gabungan, dan pembungkusan push service. Penyederhanaan itu menjaga project dua fitur tetap terbaca.</td></tr>
</table>